

Expressions and Immutable Data

CS 350

Dr. Joseph Eremondi

Last updated: January 7, 2025

Overview

Programming in CS 350

Racket

Flit

Types in Flit

THESE ARE YOUR
FATHER'S PARENTHESSES



ELEGANT
WEAPONS



FOR A MORE... CIVILIZED AGE.

Overview

Objectives

- To learn how to use Dr. Racket

Objectives

- To learn how to use Dr. Racket
- To learn how to install and use `#lang flit`

Programming in CS 350

All coding for this class uses:

- The Racket Programming Language

All coding for this class uses:

- The Racket Programming Language
- The `flit` library for Racket

All coding for this class uses:

- The Racket Programming Language
- The `flit` library for Racket
 - Is it a library? Is it a language?

All coding for this class uses:

- The Racket Programming Language
- The `flit` library for Racket
 - Is it a library? Is it a language?
 - Yes

All coding for this class uses:

- The Racket Programming Language
- The `flit` library for Racket
 - Is it a library? Is it a language?
 - Yes
- The Dr. Racket editor

Racket

What is Racket?

- Lisp-style language

What is Racket?

- Lisp-style language
 - (((((((((Parentheses))))))))

What is Racket?

- Lisp-style language
 - (((((((((Parentheses))))))))
- Language for making languages

What is Racket?

- Lisp-style language
 - (((((((((Parentheses))))))))
- Language for making languages
 - You will find documentation for many of these languages online

What is Racket?

- Lisp-style language
 - (((((((((Parentheses))))))))
- Language for making languages
 - You will find documentation for many of these languages online
 - They are different from what we are doing. Beware!

What is Dr. Racket?

- IDE for Racket

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching
 - Other useful features

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching
 - Other useful features
 - Some I've developed custom for this class

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching
 - Other useful features
 - Some I've developed custom for this class
- Read-Eval-Print-Loop (REPL)

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching
 - Other useful features
 - Some I've developed custom for this class
- Read-Eval-Print-Loop (REPL)
 - Feedback when writing code

What is Dr. Racket?

- IDE for Racket
 - Syntax highlighting
 - Parentheses matching
 - Other useful features
 - Some I've developed custom for this class
- Read-Eval-Print-Loop (REPL)
 - Feedback when writing code
 - Can evaluate expressions while you're writing your code

Flit

Objectives

- Learn the syntax of core flit features

Objectives

- Learn the syntax of core flit features
 - Numbers, booleans, if, functions, let*

Objectives

- Learn the syntax of core flit features
 - Numbers, booleans, if, functions, let*
- Learn the semantics of these features

Objectives

- Learn the syntax of core flit features
 - Numbers, booleans, if, functions, let*
- Learn the semantics of these features
 - First view of equations for understanding programs

Objectives

- Learn the syntax of core flit features
 - Numbers, booleans, `if`, functions, `let*`
- Learn the semantics of these features
 - First view of equations for understanding programs
- Understand how functional `if` differs from imperative `if`

What is Flit?

- “Functional Languages, Interpreters and Types”

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types
 - Type annotations for functions

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types
 - Type annotations for functions
 - Minimal

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types
 - Type annotations for functions
 - Minimal
 - Has what you need to write programming languages

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types
 - Type annotations for functions
 - Minimal
 - Has what you need to write programming languages
 - Not much else

What is Flit?

- “Functional Languages, Interpreters and Types”
- Language defined in Racket
 - Functions you can call
 - Adds syntax to Racket
 - Declaring and pattern matching on data types
 - Type annotations for functions
 - Minimal
 - Has what you need to write programming languages
 - Not much else
 - You can do a lot with very little

Flit features:

- Purely functional

Flit features:

- Purely functional
 - Variables never change values

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference
 - Don't have to write down the types

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference
 - Don't have to write down the types
 - Writing types down is still a good idea

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference
 - Don't have to write down the types
 - Writing types down is still a good idea
 - Compiler checks if the actual type matches what you wrote

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference
 - Don't have to write down the types
 - Writing types down is still a good idea
 - Compiler checks if the actual type matches what you wrote
 - Serves as documentation

Flit features:

- Purely functional
 - Variables never change values
 - Can't do `int x = 3; x = 4;`
- Strongly typed
 - Every expression has a type
 - No implicit coercion between types
- Type inference
 - Don't have to write down the types
 - Writing types down is still a good idea
 - Compiler checks if the actual type matches what you wrote
 - Serves as documentation
- Data Types with Variants

- Expression: something that we run to compute a value

- Expression: something that we run to compute a value
- Every flit expression is either:

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. `3`, `"hello"`, `#t`, `#f`)

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)
- Parentheses mark the start and end of complex expressions

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)
- Parentheses mark the start and end of complex expressions
- If the first thing in the parentheses is a keyword, then that keyword determines the meaning of the expression

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)
- Parentheses mark the start and end of complex expressions
- If the first thing in the parentheses is a keyword, then that keyword determines the meaning of the expression
 - e.g. if, lambda, type-case

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. `3`, `"hello"`, `#t`, `#f`)
 - An identifier (variable/function name), e.g. `x`, `y`, `+`, `and`, `or`
 - Parentheses containing some expression (`(a b c d ...)`)
- Parentheses mark the start and end of complex expressions
- If the first thing in the parentheses is a keyword, then that keyword determines the meaning of the expression
 - e.g. `if`, `lambda`, `type-case`
- Otherwise, interpreted as a function call

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)
- Parentheses mark the start and end of complex expressions
- If the first thing in the parentheses is a keyword, then that keyword determines the meaning of the expression
 - e.g. if, lambda, type-case
- Otherwise, interpreted as a function call
 - First thing in the parentheses is the function to be called

- Expression: something that we run to compute a value
- Every flit expression is either:
 - A literal (e.g. 3, "hello", #t, #f)
 - An identifier (variable/function name), e.g. x, y, +, and, or
 - Parentheses containing some expression (a b c d ...)
- Parentheses mark the start and end of complex expressions
- If the first thing in the parentheses is a keyword, then that keyword determines the meaning of the expression
 - e.g. if, lambda, type-case
- Otherwise, interpreted as a function call
 - First thing in the parentheses is the function to be called
 - NOT always a named function!

- All operations in Flit are written using “prefix notation”

- All operations in Flit are written using “prefix notation”
 - Operation, then arguments

- All operations in Flit are written using “prefix notation”
 - Operation, then arguments
 - $(+ \ 2 \ 3)$, not $(2 \ + \ 3)$

Infix vs. Prefix

- All operations in Flit are written using “prefix notation”
 - Operation, then arguments
 - $(+ \ 2 \ 3)$, not $(2 \ + \ 3)$
- Languages like C++ and Python use “infix notation” for math

Function calls

- In C++ you write `f(x, y, z)`

Function calls

- In C++ you write `f(x, y, z)`
- In Flit, you write `( f x y z )`

Function calls

- In C++ you write `f(x, y, z)`
- In Flit, you write `(f x y z)`
- 99% of what you do in Flit is calling functions

Why Parentheses?

- They're different from what you're used to

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends
- You can tell exactly the order of operations

Why Parentheses?

- They're different from what you're used to
 - I want to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends
- You can tell exactly the order of operations
 - No need for BEDMAS/PEMDAS or precedence rules

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends
- You can tell exactly the order of operations
 - No need for BEDMAS/PEMDAS or precedence rules
- Most importantly: **Programs are Trees**

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends
- You can tell exactly the order of operations
 - No need for BEDMAS/PEMDAS or precedence rules
- Most importantly: **Programs are Trees**
 - Hierarchical structure: expressions contain expressions contain expressions...

Why Parentheses?

- They're different from what you're used to
 - I want to to learn to see *past* syntax
- You can tell *exactly* where an expression starts and ends
- You can tell exactly the order of operations
 - No need for BEDMAS/PEMDAS or precedence rules
- Most importantly: **Programs are Trees**
 - Hierarchical structure: expressions contain expressions contain expressions...
 - Parentheses make this very explicit

Square vs. Round vs. Curly

- Racket treats all parentheses as the same

Square vs. Round vs. Curly

- Racket treats all parentheses as the same

Square vs. Round vs. Curly

- Racket treats all parentheses as the same

```
( + 1 2 )
```

```
[ + 1 2 ]
```

```
{ + 1 2 }
```

Square vs. Round vs. Curly

- Racket treats all parentheses as the same

```
( + 1 2 )  
[ + 1 2 ]  
{ + 1 2 }
```

```
3  
3  
3
```

- We'll use square brackets in some places for clarity

Square vs. Round vs. Curly

- Racket treats all parentheses as the same

```
( + 1 2 )  
[ + 1 2 ]  
{ + 1 2 }
```

```
3  
3  
3
```

- We'll use square brackets in some places for clarity
 - Save Curly brackets for the second half of the class

Parentheses in the real world

- e.g. Clojure, uses parentheses

Parentheses in the real world

- e.g. Clojure, uses parentheses
- WebAssembly has an S-Expression based syntax for programmers

Parentheses in the real world

- e.g. Clojure, uses parentheses
- WebAssembly has an S-Expression based syntax for programmers
- JSON is basically just funny-looking parentheses

Numbers

- Most operations prefix version of what you've seen in C++/python

Numbers

- Most operations prefix version of what you've seen in C++/python

Numbers

- Most operations prefix version of what you've seen in C++/python

```
(+ 2 7)
(- 10 0.5)
(* 1/3 2/3)
(/ 1 1000000000000000.0)
(max 10 20)
(modulo 10 3)
```

Numbers

- Most operations prefix version of what you've seen in C++/python

```
(+ 2 7)
(- 10 0.5)
(* 1/3 2/3)
(/ 1 1000000000000000.0)
(max 10 20)
(modulo 10 3)
```

```
9
9.5
2/9
1e-12
20
1
```

Booleans

- Literals written `#t` and `#f`

Booleans

- Literals written `#t` and `#f`
- Prefix versions of most boolean operations

Booleans

- Literals written `#t` and `#f`
- Prefix versions of most boolean operations

Booleans

- Literals written #t and #f
- Prefix versions of most boolean operations

```
(= (+ 2 3) 5)
(> (/ 0 1) 1)
(zero? (- (+ 1 2) (+ 3 0)))
(and (< 1 2) (> 1 0))
(or (zero? 1) (even? 3))
```


Booleans

- Literals written #t and #f
- Prefix versions of most boolean operations

```
(= (+ 2 3) 5)
(> (/ 0 1) 1)
(zero? (- (+ 1 2) (+ 3 0)))
(and (< 1 2) (> 1 0))
(or (zero? 1) (even? 3))
```

```
#t
#f
#t
#t
#f
```

Imperative Languages

- C, C++, and Python are *imperative languages*

Imperative Languages

- C, C++, and Python are *imperative languages*
 - Each statement tells what to *do*

Imperative Languages

- C, C++, and Python are *imperative languages*
 - Each statement tells what to *do*
- Mutable state and side-effects

Imperative Languages

- C, C++, and Python are *imperative languages*
 - Each statement tells what to *do*
- Mutable state and side-effects
 - Program state: current variable values

Imperative Languages

- C, C++, and Python are *imperative languages*
 - Each statement tells what to *do*
- Mutable state and side-effects
 - Program state: current variable values
 - Statements can change the state

Imperative Languages

- C, C++, and Python are *imperative languages*
 - Each statement tells what to *do*
- Mutable state and side-effects
 - Program state: current variable values
 - Statements can change the state
 - *Mutate* value of variable

- Flit is *purely functional*

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*
 - These evaluate to a *value*

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*
 - These evaluate to a *value*
- Variables are *immutable*

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*
 - These evaluate to a *value*
- Variables are *immutable*
 - Like variables in math

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*
 - These evaluate to a *value*
- Variables are *immutable*
 - Like variables in math
 - Can make new ones, but value never changes

Functional Language

- Flit is *purely functional*
- Program is made up of *expressions*
 - These evaluate to a *value*
- Variables are *immutable*
 - Like variables in math
 - Can make new ones, but value never changes
- Use functions and recursion to do all computation

Example: Imperative Conditionals

- Conditionals in C++/Python tell what to *do*

Example: Imperative Conditionals

- Conditionals in C++/Python tell what to *do*

Example: Imperative Conditionals

- Conditionals in C++/Python tell what to *do*

```
if (someCondition)
  doThis
else
  doThat
```

- The if-statement doesn't have a value, but it can mutate (change) the state

Conditional Expressions

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
    3  
    40))
```

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
       3  
       40))
```

```
"hello"
```

```
6
```

- Conditionals in Flit are **expressions**, not statements

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
    3  
    40))
```

"hello"

6

- Conditionals in Flit are **expressions**, not statements
- Value is either the “then” branch or the “else” branch

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
    3  
    40))
```

"hello"

6

- Conditionals in Flit are **expressions**, not statements
- Value is either the “then” branch or the “else” branch
 - Always have 2 branches

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
        3  
        40))
```

"hello"

6

- Conditionals in Flit are **expressions**, not statements
- Value is either the “then” branch or the “else” branch
 - Always have 2 branches
- Boolean changes what the expression **is**, not what it does

Conditional Expressions

```
(if (< 2 3) "hello" "goodbye")  
;; another example  
(+ 3  
  (if (= 2 (+ 1 1))  
    3  
    40))
```

"hello"

6

- Conditionals in Flit are **expressions**, not statements
- Value is either the “then” branch or the “else” branch
 - Always have 2 branches
- Boolean changes what the expression **is**, not what it does
 - e.g. Above, the 2nd if evaluates to 3 because the condition evaluates to true

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.
- *Referential transparency*

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.
- *Referential transparency*
 - If you give a functional program the same inputs, you will always get the same outputs

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.
- *Referential transparency*
 - If you give a functional program the same inputs, you will always get the same outputs
- Example:

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.
- *Referential transparency*
 - If you give a functional program the same inputs, you will always get the same outputs
- Example:

Definition (Semantics of +)

For any number literals m and n , the expression $(+ \ m \ n)$ is equal to the sum of m and n . The other arithmetic operations have similar rules.

Equational Reasoning

- When all variables are immutable, we can treat programs as mathematical objects
- Write equations about programs as if they were numbers/booleans/etc.
- *Referential transparency*
 - If you give a functional program the same inputs, you will always get the same outputs
- Example:

Definition (Semantics of +)

For any number literals m and n , the expression $(+ \ m \ n)$ is equal to the sum of m and n . The other arithmetic operations have similar rules.

- Math plus is different than Flit plus, needs numbers, not expressions

Imperative Breaks Equational Reasoning

- In C++, the equals sign is a LIE

Imperative Breaks Equational Reasoning

- In C++, the equals sign is a LIE

Imperative Breaks Equational Reasoning

- In C++, the equals sign is a LIE

```
int x;  
x = 3;  
x = 4;
```

- Have $x = 3$ and $x = 4$, so surely $3 = 4$, right?

Equations for IF

- We can describe the behaviour of if-expressions with the following two equations:

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions EXPR1 and EXPR2

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions EXPR1 and EXPR2

- $(\text{if } \#t \text{ EXPR1 EXPR2}) = \text{EXPR1}$

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions EXPR1 and EXPR2

- $(\text{if } \#t \text{ EXPR1 EXPR2}) = \text{EXPR1}$
- $(\text{if } \#f \text{ EXPR1 EXPR2}) = \text{EXPR2}$

Equations for IF

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions $EXPR1$ and $EXPR2$

- $(\text{if } \#t \text{ } EXPR1 \text{ } EXPR2) = EXPR1$
- $(\text{if } \#f \text{ } EXPR1 \text{ } EXPR2) = EXPR2$

- The equals sign is not a lie here

Equations for IF

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions $EXPR1$ and $EXPR2$

- $(\text{if } \#t \text{ } EXPR1 \text{ } EXPR2) = EXPR1$
- $(\text{if } \#f \text{ } EXPR1 \text{ } EXPR2) = EXPR2$

- The equals sign is not a lie here
 - The two sides of the equation have the same run-time result

- We can describe the behaviour of if-expressions with the following two equations:

Definition (Semantics of If)

For any expressions EXPR1 and EXPR2

- $(\text{if } \#t \text{ EXPR1 EXPR2}) = \text{EXPR1}$
- $(\text{if } \#f \text{ EXPR1 EXPR2}) = \text{EXPR2}$

- The equals sign is not a lie here
 - The two sides of the equation have the same run-time result
 - No matter what we choose for EXPR1 or EXPR2

- Calling a function replaces variable with concrete argument

- Calling a function replaces variable with concrete argument

Functions

- Calling a function replaces variable with concrete argument

```
;; Define a function  
(define (addOne [x : Number]) : Number  
  (+ x 1))  
  
;; Call the function we defined  
(addOne 10)
```

- Calling a function replaces variable with concrete argument

```
;; Define a function  
(define (addOne [x : Number]) : Number  
  (+ x 1))  
  
;; Call the function we defined  
(addOne 10)
```

Calling Functions

- Syntax for calling a function is

Calling Functions

- Syntax for calling a function is
 - Open bracket

Calling Functions

- Syntax for calling a function is
 - Open bracket
 - Function name

Calling Functions

- Syntax for calling a function is
 - Open bracket
 - Function name
 - Arguments, each separated by whitespace

Calling Functions

- Syntax for calling a function is
 - Open bracket
 - Function name
 - Arguments, each separated by whitespace
 - Close bracket

Calling Functions

- Syntax for calling a function is
 - Open bracket
 - Function name
 - Arguments, each separated by whitespace
 - Close bracket

Calling Functions

- Syntax for calling a function is
 - Open bracket
 - Function name
 - Arguments, each separated by whitespace
 - Close bracket

```
(f arg1 arg2 arg3 ... argN)
```

- Each argument might be a variable or a literal, or a complex expression in brackets

Defining Multi-Argument Functions

Defining Multi-Argument Functions

```
(define (isRemainder [x : Number]
                  [y : Number]
                  [remainder : Number])
  : Boolean
  (= remainder (modulo x y)))
(isRemainder 10 3 1)
(isRemainder 10 4 1)
```

Defining Multi-Argument Functions

```
(define (isRemainder [x : Number]
                    [y : Number]
                    [remainder : Number])
  : Boolean
  (= remainder (modulo x y)))
(isRemainder 10 3 1)
(isRemainder 10 4 1)
```

```
#t
```

```
#f
```

General Form Defining Functions

- General form:

General Form Defining Functions

- General form:

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments

General Form Defining Functions

- General form:

```
(define (functionName  
  [argName : argType]  
  ...  
  [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]
 - We'll use : to mean “has type” a lot in this class

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]
 - We'll use : to mean “has type” a lot in this class
- After name/arguments closing paren, have : returnType

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]
 - We'll use : to mean “has type” a lot in this class
- After name/arguments closing paren, have : returnType
- Then, the body of the function

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]
 - We'll use : to mean “has type” a lot in this class
- After name/arguments closing paren, have : returnType
- Then, the body of the function
 - e.g. the code to run

General Form Defining Functions

- General form:

```
(define (functionName  
    [argName : argType]  
    ...  
    [argNameN : argTypeN]) : returnType  
  functionBody)
```

- Brackets around the function name, followed by arguments
 - Each argument given as [name : Type]
 - We'll use : to mean “has type” a lot in this class
- After name/arguments closing paren, have : returnType
- Then, the body of the function
 - e.g. the code to run
- Later in the course we'll see another way of defining functions

- We can leave types off function arguments and return type

Type Inference

- We can leave types off function arguments and return type
- Compiler is *usually* smart enough to figure out the types

Type Inference

- We can leave types off function arguments and return type
- Compiler is *usually* smart enough to figure out the types

Type Inference

- We can leave types off function arguments and return type
- Compiler is *usually* smart enough to figure out the types

```
;; example
```

```
(define (addone x)  
  (+ x 1))
```

```
;; in general
```

```
(define (functionName argName1 argName2 ... argNameN)  
  functionBody)
```

- Functions are about *substitution*:

Semantics of Function Calls

- Functions are about *substitution*:

Definition (Semantics of Function Calls)

If a function f is defined as:

then for any expression $EXPR$, we have:

where $[x \Rightarrow EXPR]body$ is $body$, with all non-shadowed occurrences of the variable x replaced by $EXPR$

Semantics of Function Calls

- Functions are about *substitution*:

Definition (Semantics of Function Calls)

If a function f is defined as:

- $(\text{define } (f\ x)\ \text{body})$

then for any expression EXPR , we have:

where $[x \Rightarrow \text{EXPR}]\text{body}$ is body , with all non-shadowed occurrences of the variable x replaced by EXPR

Semantics of Function Calls

- Functions are about *substitution*:

Definition (Semantics of Function Calls)

If a function f is defined as:

- $(\text{define } (f \ x) \ \text{body})$

then for any expression EXPR , we have:

- $(f \ \text{EXPR}) = [x \Rightarrow \text{EXPR}] \text{body}$

where $[x \Rightarrow \text{EXPR}] \text{body}$ is body , with all non-shadowed occurrences of the variable x replaced by EXPR

Intermediate definitions

- Can still define variables

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes
 - Allows re-use

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes
 - Allows re-use
 - Only evaluated once, can use multiple times

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes
 - Allows re-use
 - Only evaluated once, can use multiple times

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes
 - Allows re-use
 - Only evaluated once, can use multiple times

```
(define (squaredSum [x : Number]
                  [y : Number]) : Number
  (let ([xy (+ x y)])
    (* xy xy)))
(squaredSum 1 2)
```

Intermediate definitions

- Can still define variables
 - Once they're given a value, never changes
 - Allows re-use
 - Only evaluated once, can use multiple times

```
(define (squaredSum [x : Number]
                  [y : Number]) : Number
  (let ([xy (+ x y)])
    (* xy xy)))
(squaredSum 1 2)
```

Understanding Definitions

In C++, you might write

Understanding Definitions

In C++, you might write

Understanding Definitions

In C++, you might write

```
while (someCond){  
    int x = f(3);  
    doSomethingWith(x);  
    doSomethingElseWith(x);  
}
```

- let is similar

Understanding Definitions

In C++, you might write

```
while (someCond){  
    int x = f(3);  
    doSomethingWith(x);  
    doSomethingElseWith(x);  
}
```

- let is similar
 - Scope is clear from the brackets

Understanding Definitions

In C++, you might write

```
while (someCond){  
    int x = f(3);  
    doSomethingWith(x);  
    doSomethingElseWith(x);  
}
```

- let is similar
 - Scope is clear from the brackets
 - Functional, so the doSomething never changes the value of x

Understanding Definitions

In C++, you might write

```
while (someCond){  
    int x = f(3);  
    doSomethingWith(x);  
    doSomethingElseWith(x);  
}
```

- let is similar
 - Scope is clear from the brackets
 - Functional, so the doSomething never changes the value of x
 - Could have just written f(3) instead of x in both places

Understanding Definitions

In C++, you might write

```
while (someCond){  
    int x = f(3);  
    doSomethingWith(x);  
    doSomethingElseWith(x);  
}
```

- let is similar
 - Scope is clear from the brackets
 - Functional, so the doSomething never changes the value of x
 - Could have just written f(3) instead of x in both places
 - Same, as long as there's no side effects

- In general:

- In general:

General Syntax

- In general:

```
(let* ([varName1  EXPR1]  
       [varName2  EXPR2]  
       ...  
       [varNameN  EXPRN])  
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body

General Syntax

- In general:

```
(let* ([varName1 EXPR1]  
       [varName2 EXPR2]  
       ...  
       [varNameN EXPRN])  
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body
- Defined vars `[variableName variableValue]`

General Syntax

- In general:

```
(let* ([varName1 EXPR1]
      [varName2 EXPR2]
      ...
      [varNameN EXPRN])
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body
- Defined vars `[variableName variableValue]`
 - Round vs. Square not important

General Syntax

- In general:

```
(let* ([varName1 EXPR1]
        [varName2 EXPR2]
        ...
        [varNameN EXPRN])
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body
- Defined vars `[variableName variableValue]`
 - Round vs. Square not important
 - Defining just one variable has double brackets

General Syntax

- In general:

```
(let* ([varName1 EXPR1]
      [varName2 EXPR2]
      ...
      [varNameN EXPRN])
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body
- Defined vars `[variableName variableValue]`
 - Round vs. Square not important
 - Defining just one variable has double brackets
 - `(let* ([x 3])) (+ x 1))`

General Syntax

- In general:

```
(let* ([varName1 EXPR1]  
       [varName2 EXPR2]  
       ...  
       [varNameN EXPRN])  
  bodyEXPR)
```

- Keyword `let*`, a bracketed list of defined variables, then the body
- Defined vars `[variableName variableValue]`
 - Round vs. Square not important
 - Defining just one variable has double brackets
 - `(let* ([x 3])) (+ x 1))`
 - Kind of awkward, but just how it works

Definition (Semantics of Let for Single Variables)

For any expression EXPR , we have:

where $[x \Rightarrow \text{EXPR}] \text{body}$ is body with all non-shadowed occurrences of x replaced by EXPR

Definition (Semantics of Let for Single Variables)

For any expression $EXPR$, we have:

- $(\text{let } ([x \text{ } EXPR]) \text{ body}) = [x \Rightarrow EXPR] \text{ body}$

where $[x \Rightarrow EXPR] \text{ body}$ is body with all non-shadowed occurrences of x replaced by $EXPR$

Definition (Semantics of Let for Single Variables)

For any expression $EXPR$, we have:

- $(\text{let } ([x \text{ } EXPR]) \text{ body}) = [x \Rightarrow EXPR] \text{ body}$

where $[x \Rightarrow EXPR] \text{ body}$ is body with all non-shadowed occurrences of x replaced by $EXPR$

- Multiple variables work similarly, but also do the substitution in the expressions of later variables values

Definition (Semantics of Let for Single Variables)

For any expression $EXPR$, we have:

- $(\text{let } ([x \text{ } EXPR]) \text{ body}) = [x \Rightarrow EXPR] \text{ body}$

where $[x \Rightarrow EXPR] \text{ body}$ is body with all non-shadowed occurrences of x replaced by $EXPR$

- Multiple variables work similarly, but also do the substitution in the expressions of later variables values
 - We'll see this formally later

Alternate versions of Let

- There's also `let`

Alternate versions of Let

- There's also `let`
 - Like `let*`, but when multiple variables, can't refer to each other

Alternate versions of Let

- There's also `let`
 - Like `let*`, but when multiple variables, can't refer to each other
 - Recursive version `let rec`

Alternate versions of Let

- There's also `let`
 - Like `let*`, but when multiple variables, can't refer to each other
 - Recursive version `let rec`
 - Can define multiple variables that all refer to each other and themselves

Alternate versions of Let

- There's also `let`
 - Like `let*`, but when multiple variables, can't refer to each other
 - Recursive version `let rec`
 - Can define multiple variables that all refer to each other and themselves
- For now you only need `let*`

Alternate versions of Let

- There's also `let`
 - Like `let*`, but when multiple variables, can't refer to each other
 - Recursive version `let rec`
 - Can define multiple variables that all refer to each other and themselves
- For now you only need `let*`
 - Very useful for gradually building up a complex solution from previous smaller bits

Types in Flit

Why write down types?

- With type inference, we don't *need* to write down types

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code
- Documentation

Why write down types?

- With type inference, we don't *need* to write down types
- Still a good idea to write them down
- Writing a type is like running an infinite number of tests
 - Compiler makes sure that every possible run of your program produces the type written down
- Sanity check
 - Make sure types in your head match the types in the code
- Documentation
 - Types make it very clear what arguments a function expects

Checking Types

- Can annotate any expression to confirm its type

Checking Types

- Can annotate any expression to confirm its type

Checking Types

- Can annotate any expression to confirm its type

```
(has-type 3 : Number)  
(has-type #t : Boolean)  
(has-type "hello" : String)
```

Checking Types

- Can annotate any expression to confirm its type

```
(has-type 3 : Number)
(has-type #t : Boolean)
(has-type "hello" : String)
```

```
3
#t
"hello"
```

Thinking About Types

- With any type, there are two things we care about

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type
 - Can also create/use any value by giving it to a function expecting/returning that type

Thinking About Types

- With any type, there are two things we care about
 - How to *directly* create a value of that type
 - How to *directly* use a value of that type
 - Can also create/use any value by giving it to a function expecting/returning that type
 - But eventually, you need some primitive operations

- To create: literals 3, 22, 0.00001

- To create: literals 3, 22, 0.00001
- To use: built in arithmetic +, -, *, / etc. and comparisons like zero?

- To use: mostly built-in operations

- To use: mostly built-in operations
 - I'll highlight relevant functions as we see them

- To create: literals `#t`, `#f`

Booleans

- To create: literals `#t`, `#f`
- To use: `if someBool e1 e2`

Booleans

- To create: literals `#t`, `#f`
- To use: `if someBool e1 e2`
- E.g. the purpose of booleans is to be things we give to `if` expressions

Types for If

- `if` is not a function

- `if` is not a function
 - We'll learn exactly why later

Types for If

- `if` is not a function
 - We'll learn exactly why later
- Need a special rule to determine its type

- `if` is not a function
 - We'll learn exactly why later
- Need a special rule to determine its type
 - Can have any type, depending on the type you give it

Types for If

- `if` is not a function
 - We'll learn exactly why later
- Need a special rule to determine its type
 - Can have any type, depending on the type you give it
- Just like we have equations, we can make formal rules for typing

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$
- Condition must always be boolean

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**
 - Type-checker will give error if branches have different types

Typing Rule for If

Definition (Typing for If)

For any expressions EXPR1 , EXPR2 and EXPR3 ,

- if $\text{EXPR1} : \text{Boolean}$, and
- there is some type T such that $\text{EXPR2} : T$ and $\text{EXPR3} : T$,

then:

- $(\text{if } \text{EXPR1 } \text{EXPR2 } \text{EXPR3}) : T$

- Condition must always be boolean
- Don't care what the type of the branches is, but the **need to be the same**
 - Type-checker will give error if branches have different types
- Result of the `if` has the same type as the branches

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

- if $\text{EXPR1} : \text{Number}$

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

- if $\text{EXPR1} : \text{Number}$
- and $\text{EXPR2} : \text{Number}$

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

- if $\text{EXPR1} : \text{Number}$
- and $\text{EXPR2} : \text{Number}$
- then $(+ \text{EXPR1} \text{EXPR2}) : \text{Number}$

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

- if $\text{EXPR1} : \text{Number}$
 - and $\text{EXPR2} : \text{Number}$
 - then $(+ \text{EXPR1} \text{EXPR2}) : \text{Number}$
-
- E.g. the type checker will make sure that both inputs to $+$ are numbers, and that you use the result in a context expecting a number

Definition (Typing for Plus)

For any expressions EXPR1 , EXPR2

- if $\text{EXPR1} : \text{Number}$
 - and $\text{EXPR2} : \text{Number}$
 - then $(+ \text{EXPR1} \text{EXPR2}) : \text{Number}$
-
- E.g. the type checker will make sure that both inputs to $+$ are numbers, and that you use the result in a context expecting a number
 - Can make rules for all expressions this way

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_arg

- if $\text{EXPR_f} : T_1 \rightarrow T_2$
- and $\text{EXPR_arg} : T_1$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
- and $\text{EXPR}_{\text{arg}} : T_1$
- then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
- and $\text{EXPR}_{\text{arg}} : T_1$
- then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$

- When asking “how can I make a value of this type”:

Definition (Typing for Function Calls)

For any expressions EXPR_f and EXPR_{arg}

- if $\text{EXPR}_f : T_1 \rightarrow T_2$
- and $\text{EXPR}_{\text{arg}} : T_1$
- then $(\text{EXPR}_f \text{ EXPR}_{\text{arg}}) : T_2$

- When asking “how can I make a value of this type”:
 - Can always try “call a function that returns that type”

Function Definitions

Definition (Typing for Function Definitions)

For a definition `(define (f x1 ... xN) body)`

Function Definitions

Definition (Typing for Function Definitions)

For a definition `(define (f x1 ... xN) body)`

- if `body` : `T_ret`

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
- then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \ \text{body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types
 - Instead, can assume you already have things of those types

Function Definitions

Definition (Typing for Function Definitions)

For a definition $(\text{define } (f \ x_1 \ \dots \ x_N) \text{ body})$

- if $\text{body} : T_{\text{ret}}$
 - *under the assumption that* $x_1 : T_1 \dots x_N : T_N$
 - then $f : (T_1 \ \dots \ T_N \rightarrow T_{\text{ret}})$
-
- e.g. a function type with arguments matching the x 's types, and return type matching the body
 - Function variable types are *assumptions*, not obligations
 - To *define* function, don't provide things of the argument types
 - Instead, can assume you already have things of those types
 - e.g. the parameters

- Figure out the type of the thing you're trying to write

- Figure out the type of the thing you're trying to write
 - Often, I give this to you

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type
 - Which might contain more holes

Programming Via Holes

- Figure out the type of the thing you're trying to write
 - Often, I give this to you
- Write a dummy function with that type, with TODO as the body
- Write some tests that capture what you're expecting the function to do
- Repeat until all tests pass and there's no more holes:
 - Fill in a hole with something of the right type
 - Which might contain more holes
 - Could be function call, `if`, in-scope variable, etc.

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with 0 or more holes

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?
 - Hopefully these holes are simpler

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?
 - Hopefully these holes are simpler
 - Can try to fill them with literals, variables, or other function calls

Functions for Filling Holes

- Can always fill a hole of type T by applying a function that returns type T
 - Give a hole as each argument for that function
- Now, you've replaced one hole with $\emptyset$ or more holes
 - Yay?
 - Hopefully these holes are simpler
 - Can try to fill them with literals, variables, or other function calls
- We'll see more of this with recursion and datatypes

- Maximum function

- Maximum function
 - Writing it

- Maximum function
 - Writing it
 - Reasoning about it with equations